PRACTICAL 7: System Queuing and Server Performance Modelling
a) Mathematical Formulation and Manual Calculation of Queuing Systems
b) Multi-Server Systems and Priority Queuing

Queues occur when **customers, calls, or tasks wait for service**. Server performance modeling helps **analyze waiting times, system load, and efficiency**.

Single-Server Queue (M/M/1)

- One server handles arrivals (Poisson), service time is exponential.
- Use for **simple systems like an ATM**.

Performance Measure	Formula	Meaning
Server Utilization (ρ)	$\rho = \lambda / \mu$	Fraction of time the server is busy. (0 - 1)
Probability server is idle (P_0)	$P_0 = 1 - \rho$	Chance the server is free
Average number in system (L)	$L = \rho / (1 - \rho)$	Customers in queue + being served
Average number in queue (Lq)	$Lq = \rho^2 / (1 - \rho)$	Customers waiting only
Average time in system (W)	$W = 1 / (\mu - \lambda)$	Total time a customer spends in system (0 – 10 hours)
Average waiting time in queue (Wq)	$Wq = \rho / (\mu - \lambda)$	Time spent waiting before service. (0 – 10 hours)

Multi-Server Queue (M/M/c)

- Multiple servers handle customers simultaneously.
- Use for **call centers, banks, or helpdesks** with multiple agents.

Performance Measure	Formula (Approx)	Meaning
Server Utilization per agent (ρ)	$\rho = \lambda / (c * \mu)$	Fraction of time servers are busy
Average waiting time in queue (Wq)	$Wq \approx \rho / (c * \mu * (1 - \rho))$	Time a customer waits in queue
Average time in system (W)	$W = Wq + 1 / \mu$	Queue time + service time

Priority Queue

- Some customers are **urgent**, others are regular.
- Urgent calls are served first; regular calls wait longer.
- **Key idea:**
  - **Wq_urgent < Wq_regular** → urgent waits less
  - Total utilization **ρ = (λ_urgent + λ_regular) / (c μ)**
- Helps **improve service for critical tasks**.

Performance Measure	Formula (Approx)	Meaning
Urgent calls waiting time	Wq_urgent ≈ Wq * 0.5	Urgent calls wait less than regular calls
Regular calls waiting time	Wq_regular ≈ Wq * 1.2	Regular calls wait longer
Total server utilization (ρ)	ρ = (λ_urgent + λ_regular) / (c * μ)	Fraction of time servers are busy
Stability check	ρ < 1 → Stable, ρ ≥ 1 → Unstable	Ensures system can handle load

Scenario: ATM Single Server M/M/1 Queue

A bank has **one ATM**. Customers arrive at a rate of **λ customers per hour** and the ATM serves at **μ customers per hour**. Arrivals follow a Poisson process, service times are exponential, and the queue is FIFO.

Write a Python code to:

1. Check if the ATM system is **stable**.
2. Calculate:
   - Utilization (ρ)
   - Probability ATM is idle (P_0)
   - Average number of customers in system (L)
   - Average number in queue (Lq)
   - Average waiting time in queue (Wq)
   - Average time in system (W)
3. Plot **Arrival Rate (λ) vs Waiting Time (Wq)**.
4. Comment on how waiting time changes as λ approaches μ.

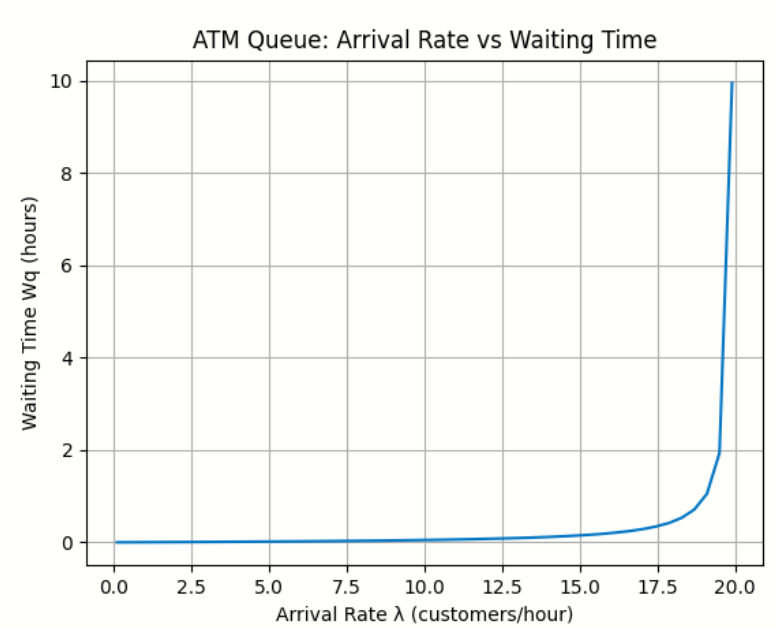
Python Code:

```
import matplotlib.pyplot as plt
import numpy as np

print("===== ATM QUEUE ANALYSIS (M/M/1) =====")

# Step 1: Get user input
lambda_rate = float(input("Enter customer arrival rate per hour (λ): "))
mu = float(input("Enter ATM service rate per hour (μ): "))

# Step 2: Calculate server utilization
# rho (ρ) = λ / μ
# Represents fraction of time ATM is busy
rho = lambda_rate / mu
print("\nServer Utilization (ρ):", round(rho, 4))

# Step 3: Check system stability
if rho >= 1:
    print("System is UNSTABLE! Too many customers for the ATM.")
else:
    print("System is STABLE.\n")

    # Step 4: Calculate performance measures
    # P0: Probability ATM is idle
    P0 = 1 - rho
    # L: Average number of customers in the system
    L = rho / (1 - rho)
    # Lq: Average number of customers waiting in queue
    Lq = rho**2 / (1 - rho)
    # W: Average time a customer spends in the system (hours)
    W = 1 / (mu - lambda_rate)
```

```
# Wq: Average waiting time in queue (hours)
Wq = rho / (mu - lambda_rate)

# Step 5: Display results
print("Probability ATM is idle (P0):", round(P0, 4))
print("Average number in system (L):", round(L, 4))
print("Average number in queue (Lq):", round(Lq, 4))
print("Average time in system (W):", round(W, 4), "hours")
print("Average waiting time in queue (Wq):", round(Wq, 4), "hours")

# Step 6: Plot Arrival Rate vs Waiting Time graph
# Create a range of arrival rates from 0.1 to just below μ
arrival_values = np.linspace(0.1, mu - 0.1, 50)
# Calculate corresponding waiting time in queue for each arrival rate
waiting_values = arrival_values / (mu * (mu - arrival_values))

# Plot simple line graph
plt.plot(arrival_values, waiting_values)
plt.xlabel("Arrival Rate λ (customers/hour)")
plt.ylabel("Waiting Time Wq (hours)")
plt.title("ATM Queue: Arrival Rate vs Waiting Time")
plt.grid(True)  # optional: adds grid lines to the graph
plt.show()
```

Result:

```
===== ATM QUEUE ANALYSIS (M/M/1) =====
Enter customer arrival rate per hour (λ): 5
Enter ATM service rate per hour (μ): 20

Server Utilization (ρ): 0.25
System is STABLE.

Probability ATM is idle (P0): 0.75
Average number in system (L): 0.3333
Average number in queue (Lq): 0.0833
Average time in system (W): 0.0667 hours
Average waiting time in queue (Wq): 0.0167 hours
```



Q2. **Title:** Performance Analysis of a Call Center with Multiple Agents

A call center has **c agents** serving incoming calls.

- Calls arrive at an average rate **λ calls per hour**.
- Each agent serves calls at **μ calls per hour**.
- If all agents are busy, calls wait in a **common queue**.
- Some calls have **priority** (urgent) and are served first.
- The system follows **FIFO within each priority level**.

Write a Python Code to:

1. Check if the system is **stable**.
2. Calculate key performance measures for **all calls**:
   - Server utilization (ρ)
   - Probability system is empty (P_0)
   - Average number of calls in system (L)
   - Average number in queue (Lq)
   - Average waiting time in queue (Wq)
   - Average time a call spends in the system (W)
3. Compare waiting time for **priority and regular calls**.
4. Plot **Arrival Rate vs Average Waiting Time**.

Python Code:

```
import matplotlib.pyplot as plt

print("===== MULTI-SERVER QUEUE (M/M/c) =====")

# ----------------------------
```

```
# Step 1: Get user input
# ----------------------------
# λ = average arrival rate (calls per hour)
lambda_rate = float(input("Enter arrival rate λ (calls/hour): "))

# μ = average service rate per agent (calls per hour)
mu = float(input("Enter service rate μ (calls/hour per agent): "))

# c = number of servers/agents
c = int(input("Enter number of agents (c): "))

# ----------------------------
# Step 2: Calculate server utilization
# ----------------------------
# ρ = fraction of time servers are busy
rho = lambda_rate / (c * mu)
print("\nServer Utilization per agent (ρ):", round(rho, 4))

# ----------------------------
# Step 3: Check if system is stable
# ----------------------------
# If utilization >= 1, the system is unstable (calls arrive faster than they can be served)
if rho >= 1:
    print("System is UNSTABLE! Too many calls for available agents.")
else:
    print("System is STABLE.\n")

    # ----------------------------
    # Step 4: Calculate performance measures
    # ----------------------------
    # Wq = average waiting time in the queue (hours)
    Wq = rho / (c * mu * (1 - rho))

    # W = average total time a customer spends in the system (queue + service)
    W = Wq + 1 / mu

    print("Average waiting time in queue (Wq):", round(Wq, 4), "hours")
    print("Average time in system (W):", round(W, 4), "hours")

    # ----------------------------
    # Step 5: Simple graph to show trend
    # ----------------------------
    # We take 4 example arrival rates (50%, 75%, 90%, 99% of current λ)
    lam1 = 0.5 * lambda_rate
    lam2 = 0.75 * lambda_rate
    lam3 = 0.9 * lambda_rate
    lam4 = 0.99 * lambda_rate
```

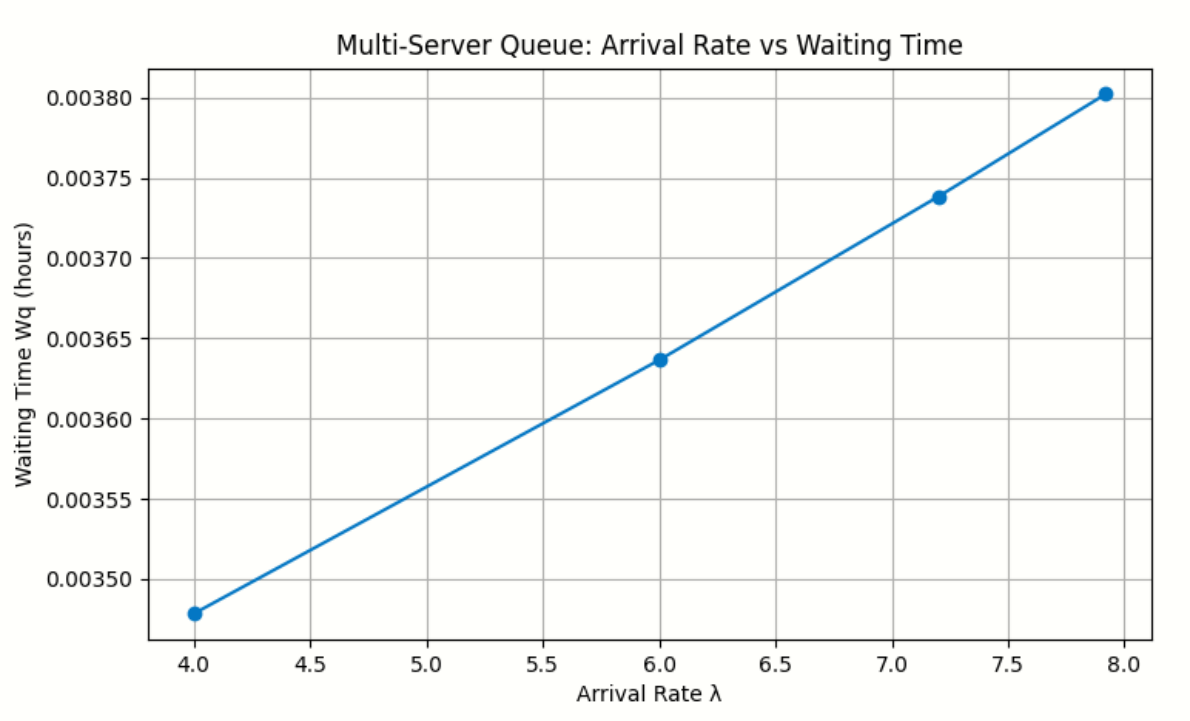
```
# Calculate waiting time for each example arrival rate
Wq1 = rho / (c * mu * (1 - lam1 / (c * mu)))
Wq2 = rho / (c * mu * (1 - lam2 / (c * mu)))
Wq3 = rho / (c * mu * (1 - lam3 / (c * mu)))
Wq4 = rho / (c * mu * (1 - lam4 / (c * mu)))

# Lists for plotting the graph
arrival_points = [lam1, lam2, lam3, lam4]
Wq_points = [Wq1, Wq2, Wq3, Wq4]

# Plot the waiting time vs arrival rate
plt.plot(arrival_points, Wq_points, marker='o')
plt.xlabel("Arrival Rate λ (calls/hour)")
plt.ylabel("Waiting Time Wq (hours)")
plt.title("Multi-Server Queue: Arrival Rate vs Waiting Time")
plt.grid(True)  # Add grid for better readability
plt.show()
```

Optional:

```
# Simple graph with 4 points
arrival_points = [0.5*lambda_rate, 0.75*lambda_rate, 0.9*lambda_rate,
0.99*lambda_rate]
Wq_points = [rho / (c * mu * (1 - (lam/(c*mu)))) for lam in arrival_points]
```

Result:

```
===== MULTI-SERVER QUEUE (M/M/c) =====
Enter arrival rate λ (calls/hour): 8
Enter service rate μ (calls/hour per agent): 10
Enter number of agents (c): 5

Server Utilization per agent (ρ): 0.16
System is STABLE.

Average waiting time in queue (Wq): 0.0038 hours
Average time in system (W): 0.1038 hours
```


Multi-Server Queue: Arrival Rate vs Waiting Time
Waiting Time Wq (hours)
0.00380
0.00375
0.00370
0.00365
0.00360
0.00355
0.00350
4.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
Arrival Rate λ

Q3. A call center has urgent and regular calls.

- Urgent calls arrive at λ_1 per hour, Regular calls arrive at λ_2 per hour.
- Each agent serves μ calls per hour.
- There are c agents.

Write a Python Code:

1. Check if the system is stable.
2. Calculate waiting times for urgent and regular calls.
3. Show a graph comparing the waiting times.

Input: λ_1, λ_2, μ, c

Output: Server utilization, waiting times, and a bar graph.

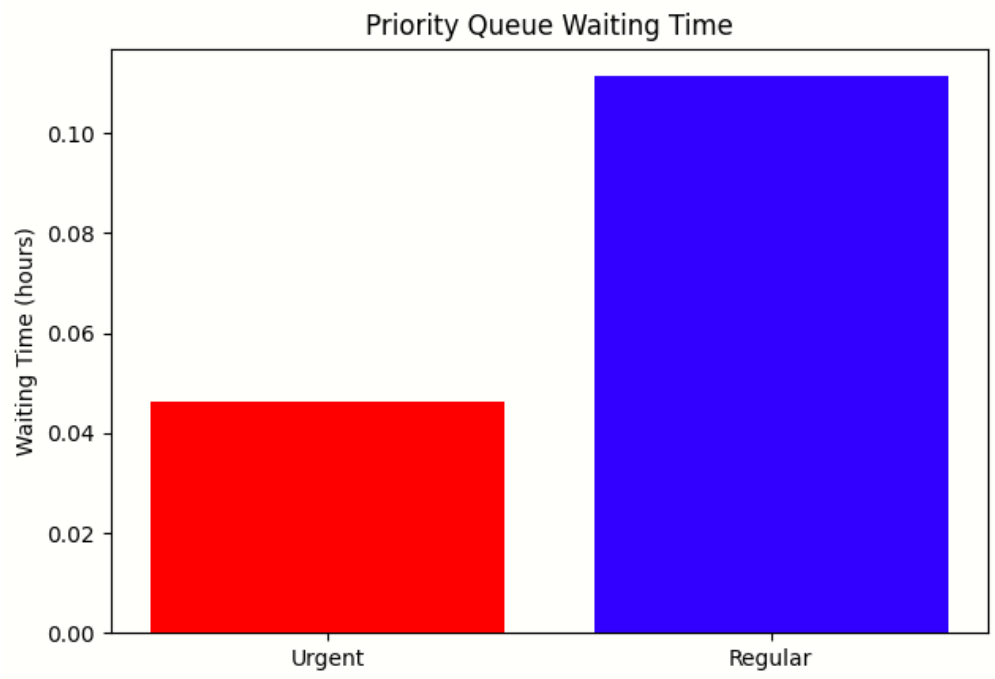
Python Code:

```
import matplotlib.pyplot as plt

print("===== PRIORITY QUEUE =====")

# ----------------------------
# Step 1: Get user input
# ----------------------------
# Urgent calls per hour
urgent = float(input("Enter urgent calls per hour: "))

# Regular calls per hour
regular = float(input("Enter regular calls per hour: "))

# Service rate per agent (calls/hour)
service = float(input("Enter service rate per agent: "))

# Number of agents
agents = int(input("Enter number of agents: "))

# ----------------------------
# Step 2: Calculate server utilization
# ----------------------------
rho = (urgent + regular) / (agents * service)  # Fraction of time servers are busy
print("\nServer Utilization ρ:", round(rho, 2))

# ----------------------------
# Step 3: Check system stability
# ----------------------------
if rho >= 1:
    print("System is UNSTABLE! Too many calls for the agents.")
else:
    print("System is STABLE.\n")

    # ----------------------------
    # Step 4: Approximate waiting times
```

```
# ----------------------------
# Urgent calls wait less
wait_urgent = (rho / (agents * service * (1 - rho))) * 0.5

# Regular calls wait more
wait_regular = (rho / (agents * service * (1 - rho))) * 1.2

print("Average waiting time for urgent calls:", round(wait_urgent, 2), "hours")
print("Average waiting time for regular calls:", round(wait_regular, 2), "hours")

# ----------------------------
# Step 5: Simple bar graph
# ----------------------------
types = ["Urgent", "Regular"]                # Labels for bars
times = [wait_urgent, wait_regular]          # Corresponding waiting times

plt.bar(types, times, color=['red','blue'])   # Create bar graph
plt.ylabel("Waiting Time (hours)")
plt.title("Priority Queue Waiting Time")
plt.show()
```

Result:

```
===== PRIORITY QUEUE =====
Enter urgent calls per hour: 5
Enter regular calls per hour: 8
Enter service rate per agent: 4
Enter number of agents: 5

Server Utilization ρ: 0.65
System is STABLE.

Average waiting time for urgent calls: 0.05 hours
Average waiting time for regular calls: 0.11 hours
```


Priority Queue Waiting Time
Waiting Time (hours)
0.00
0.02
0.04
0.06
0.08
0.10
Urgent
Regular

****